Gradient Boosting Reinforcement Learning

Benjamin Fuhrer · Chen Tessler · Gal Dalal
NVIDIA

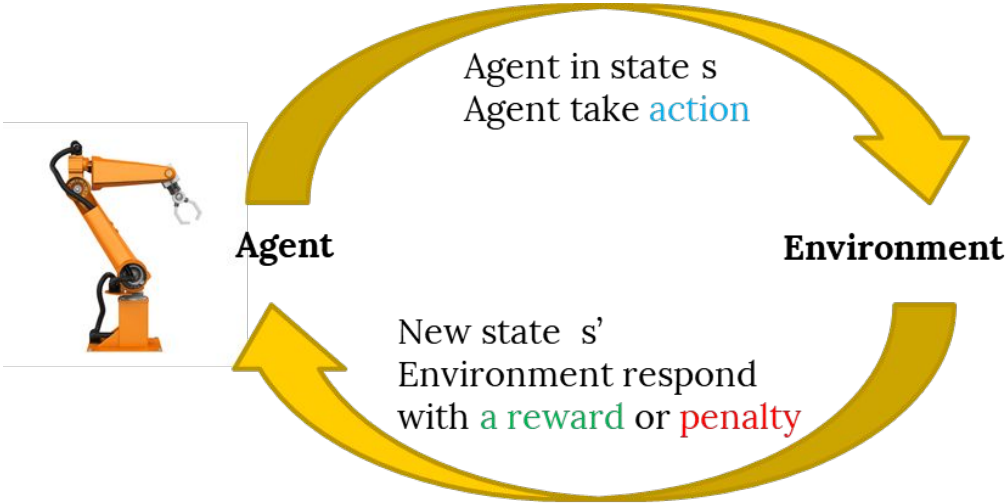
11 July 2024

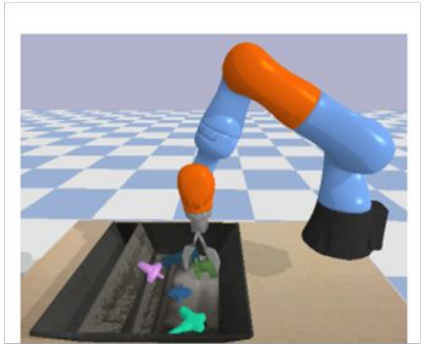
딥러닝 논문읽기 모임 펀디멘탈팀
오영화

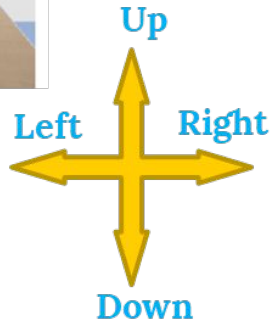
Index

Background

Reinforcement Learning





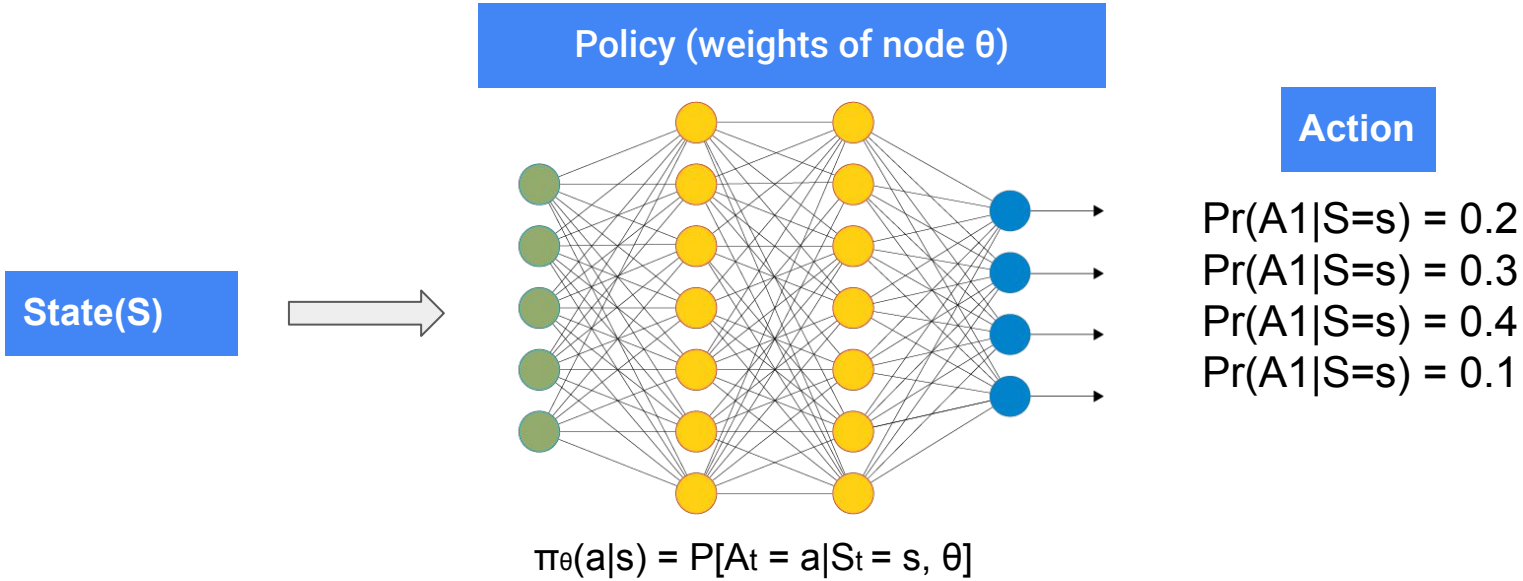


Background

Neural Network based RL

policy as a map of weight : $\theta \leftarrow \theta + \alpha \nabla \theta J(\theta)$ (α: learning rate)

Explore θ^* to maximize (minimum) objective function $J(\theta)$: $\nabla \theta J_\theta = \mathbb{E}\pi[\Sigma_a Q_\pi(S_t, a) \nabla \pi (a|S_t, \theta)]$

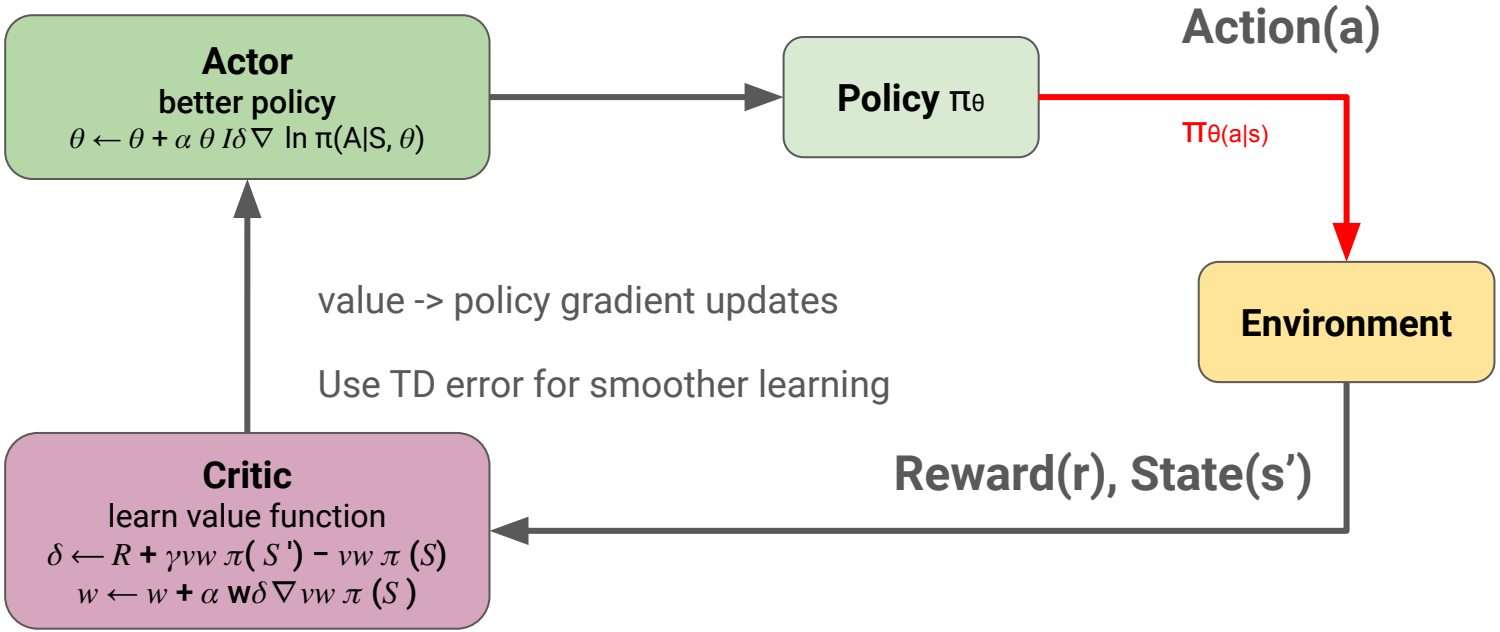


Background

Actor-Critic Reinforcement Learning

Advantage Actor Critic(A2C) RL

on-policy / reducing variance



Background

Actor-Critic Reinforcement Learning

PPO(Proximal Policy Optimization)

extends A2C by improving stability / Model-free based learning / clipped objective / prevent drastic policy changes

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)\right]$$

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t\left[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)\right]$$

c1 and c2 are coefficients.

Add an entropy bonus to ensure sufficient exploration

Squared-error value loss: $(V_\theta(s_t) - V_t^{targ})^2$

Modified version from RL — Proximal Policy Optimization (PPO) Explained by Jonathan Hui

Background

Actor-Critic Reinforcement Learning

Advantaged Weighted Regression(AWR)

Off-policy / dataset D for updating both policy and value function(replay buffer) / pre-defined(offline)

solving two regression problems

- Value function update : Minimize difference between value function and G(s,a) Montel-Carlo estimsate or TD

$$V_k = \arg\min_V \mathbb{E}_{s,a\sim D}[\|G(s,a) - V(s)\|_2^2]$$

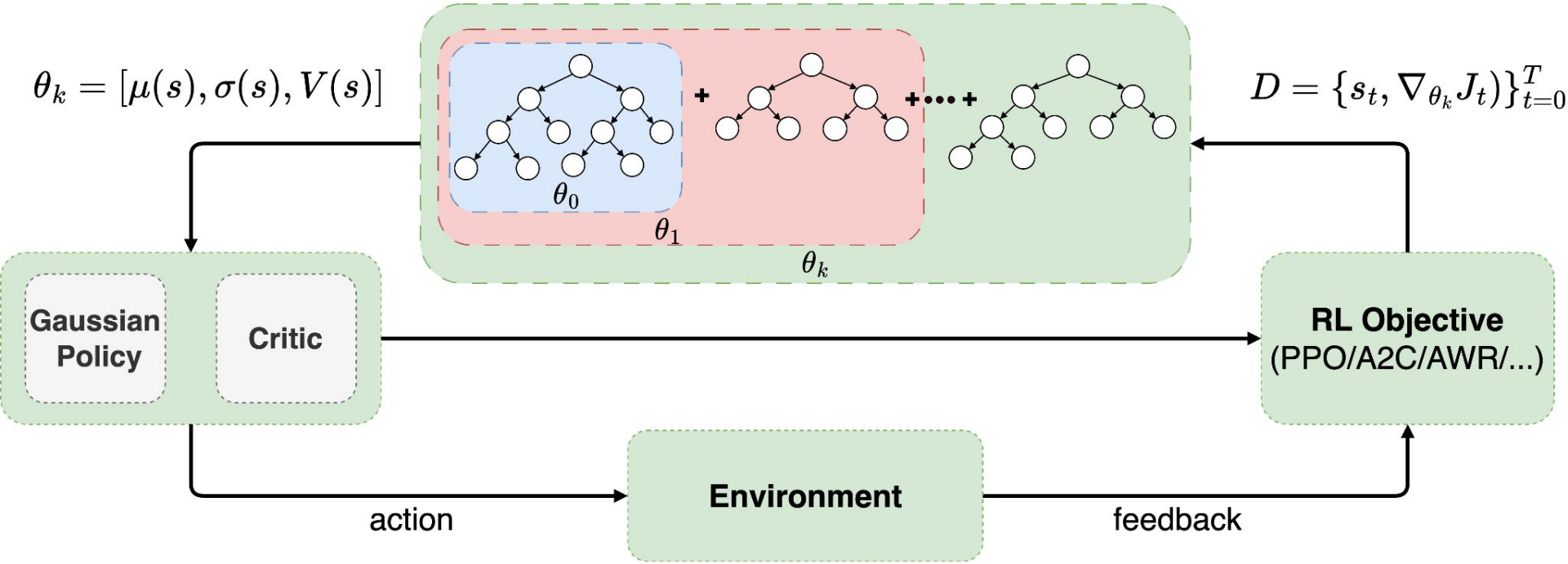
- Policy update: Maximize the weighted log probability of actions

$$\pi_{k+1} = \arg\max_\pi \mathbb{E}_{s,a\sim D}[\log \pi(a|s) \exp(\frac{1}{\beta} A_k(s,a))]$$

ANY QUESTION?

02 Gradient Boosting Reinforcement Learning

Gradient Boosting Trees & Gaussian Policy & gradient



02 Gradient Boosting Reinforcement Learning

Initialize

Algorithm 1 Gradient Boosting for Reinforcement Learning (GBRL)

Initialize: θ_0, ϵ_{actor}, ϵ_{critic}, experience buffer $\mathcal{B}$, total training iterations K, number of updates U, batch size N, $k \leftarrow 1$
while $k < K$ **do**
  Collect trajectory $\tau^{(k)} = (\mathbf{s}_0, \mathbf{a}_0, \ldots, \mathbf{s}_T, \mathbf{a}_T)^{(k)}$ and rewards $(\mathbf{r}_0, \ldots, \mathbf{r}_T)^{(k)}$ using $\pi_{\theta_{k-1}}$
  Add trajectory $\tau^{(k)}$ and rewards to the experience buffer $\mathcal{B}$
  for each update $u = 1$ to U **do**
    Sample a batch from the experience buffer $\mathcal{B}$
    Compute gradients g according to AC algorithm (e.g., PPO, A2C, AWR)
    Construct dataset $D = \{(\mathbf{s}_n, g_n)\}_{n=0}^{N}$ and fit a decision tree h_k
    for each dimension $d = 0$ to D **do**
      if $0 \leq d < D$ **then**
        Update $\theta_k^{(d)} = \theta_{k-1}^{(d)} + \epsilon_{\text{actor}} h_k(\mathbf{s})$
      else
        Update $\theta_k^{(d)} = \theta_{k-1}^{(d)} + \epsilon_{\text{critic}} h_k(\mathbf{s})$
    $k \leftarrow k + 1$
Output: AC parameters $\theta_K^{(d)}(\mathbf{s})$ for $d = 0, 1, \ldots, D$

02 Gradient Boosting Reinforcement Learning

Initialize

Algorithm 1 Gradient Boosting for Reinforcement Learning (GBRL)

Initialize: θ_0, ϵ_{actor}, ϵ_{critic}, experience buffer $\mathcal{B}$, total training iterations K, number of updates U, batch size N, $k \leftarrow 1$
while $k < K$ **do**
    Collect trajectory $\tau^{(k)} = (\mathbf{s}_0, \mathbf{a}_0, \ldots, \mathbf{s}_T, \mathbf{a}_T)^{(k)}$ and rewards $(\mathbf{r}_0, \ldots, \mathbf{r}_T)^{(k)}$ using $\pi_{\theta_{k-1}}$
    Add trajectory $\tau^{(k)}$ and rewards to the experience buffer $\mathcal{B}$
    for each update $u = 1$ to U **do**
        Sample a batch from the experience buffer $\mathcal{B}$
        Compute gradients g according to AC algorithm (e.g., PPO, A2C, AWR)
        Construct dataset $D = \{(\mathbf{s}_n, g_n)\}_{n=0}^{N}$ and fit a decision tree h_k
        for each dimension $d = 0$ to D **do**
            if $0 \leq d < D$ **then**
                Update $\theta_k^{(d)} = \theta_{k-1}^{(d)} + \epsilon_{\text{actor}} h_k(\mathbf{s})$
            else
                Update $\theta_k^{(d)} = \theta_{k-1}^{(d)} + \epsilon_{\text{critic}} h_k(\mathbf{s})$
        $k \leftarrow k + 1$
Output: AC parameters $\theta_K^{(d)}(\mathbf{s})$ for $d = 0, 1, \ldots, D$

02 Gradient Boosting Reinforcement Learning

Initialize

Algorithm 1 Gradient Boosting for Reinforcement Learning (GBRL)

Initialize: θ_0, ϵ_{actor}, ϵ_{critic}, experience buffer $\mathcal{B}$, total training iterations K, number of updates U, batch size N, $k \leftarrow 1$
while $k < K$ **do**
  Collect trajectory $\tau^{(k)} = (\mathbf{s}_0, \mathbf{a}_0, \ldots, \mathbf{s}_T, \mathbf{a}_T)^{(k)}$ and rewards $(\mathbf{r}_0, \ldots, \mathbf{r}_T)^{(k)}$ using $\pi_{\theta_{k-1}}$
  Add trajectory $\tau^{(k)}$ and rewards to the experience buffer $\mathcal{B}$
  for each update $u = 1$ to U **do**
    Sample a batch from the experience buffer $\mathcal{B}$
    Compute gradients g according to AC algorithm (e.g., PPO, A2C, AWR)
    Construct dataset $D = \{(\mathbf{s}_n, g_n)\}_{n=0}^{N}$ and fit a decision tree h_k
    for each dimension $d = 0$ to D **do**
      if $0 \leq d < D$ **then**
        Update $\theta_k^{(d)} = \theta_{k-1}^{(d)} + \epsilon_{\text{actor}} h_k(\mathbf{s})$
      else
        Update $\theta_k^{(d)} = \theta_{k-1}^{(d)} + \epsilon_{\text{critic}} h_k(\mathbf{s})$
    $k \leftarrow k + 1$
Output: AC parameters $\theta_K^{(d)}(\mathbf{s})$ for $d = 0, 1, \ldots, D$

02 Gradient Boosting Reinforcement Learning

Initialize

Algorithm 1 Gradient Boosting for Reinforcement Learning (GBRL)

Initialize: θ_0, ϵ_{actor}, ϵ_{critic}, experience buffer $\mathcal{B}$, total training iterations K, number of updates U, batch size N, $k \leftarrow 1$
while $k < K$ **do**
  Collect trajectory $\tau^{(k)} = (\mathbf{s}_0, \mathbf{a}_0, \ldots, \mathbf{s}_T, \mathbf{a}_T)^{(k)}$ and rewards $(\mathbf{r}_0, \ldots, \mathbf{r}_T)^{(k)}$ using $\pi_{\theta_{k-1}}$
  Add trajectory $\tau^{(k)}$ and rewards to the experience buffer $\mathcal{B}$
  for each update $u = 1$ to U **do**
    Sample a batch from the experience buffer $\mathcal{B}$
    Compute gradients g according to AC algorithm (e.g., PPO, A2C, AWR)
    Construct dataset $D = \{(\mathbf{s}_n, g_n)\}_{n=0}^{N}$ and fit a decision tree h_k
  for each dimension $d = 0$ to D **do**
    if $0 \leq d < D$ **then**
      Update $\theta_k^{(d)} = \theta_{k-1}^{(d)} + \epsilon_{\text{actor}} h_k(\mathbf{s})$
    else
      Update $\theta_k^{(d)} = \theta_{k-1}^{(d)} + \epsilon_{\text{critic}} h_k(\mathbf{s})$
  $k \leftarrow k + 1$
Output: AC parameters $\theta_K^{(d)}(\mathbf{s})$ for $d = 0, 1, \ldots, D$

02 Gradient Boosting Reinforcement Learning

Initialize

Algorithm 1 Gradient Boosting for Reinforcement Learning (GBRL)

Initialize: θ_0, ϵ_{actor}, ϵ_{critic}, experience buffer $\mathcal{B}$, total training iterations K, number of updates U, batch size N, $k \leftarrow 1$
while $k < K$ **do**
  Collect trajectory $\tau^{(k)} = (\mathbf{s}_0, \mathbf{a}_0, \ldots, \mathbf{s}_T, \mathbf{a}_T)^{(k)}$ and rewards $(\mathbf{r}_0, \ldots, \mathbf{r}_T)^{(k)}$ using $\pi_{\theta_{k-1}}$
  Add trajectory $\tau^{(k)}$ and rewards to the experience buffer $\mathcal{B}$
  for each update $u = 1$ to U **do**
    Sample a batch from the experience buffer $\mathcal{B}$
    Compute gradients g according to AC algorithm (e.g., PPO, A2C, AWR)
    Construct dataset $D = \{(\mathbf{s}_n, g_n)\}_{n=0}^{N}$ and fit a decision tree h_k
    for each dimension $d = 0$ to D **do**
      if $0 \leq d < D$ **then**
        Update $\theta_k^{(d)} = \theta_{k-1}^{(d)} + \epsilon_{\text{actor}} h_k(\mathbf{s})$
      else
        Update $\theta_k^{(d)} = \theta_{k-1}^{(d)} + \epsilon_{\text{critic}} h_k(\mathbf{s})$
    $k \leftarrow k + 1$
Output: AC parameters $\theta_K^{(d)}(\mathbf{s})$ for $d = 0, 1, \ldots, D$

Experiments

Experiments aim to answer the following questions

1. GBT as RL Function Approximator: Can GBT-based AC algorithms effectively solve complex high-dimensional RL tasks?

2. Comparison to NNs: How does GBRL compare with NN-based training in various RL algorithms?

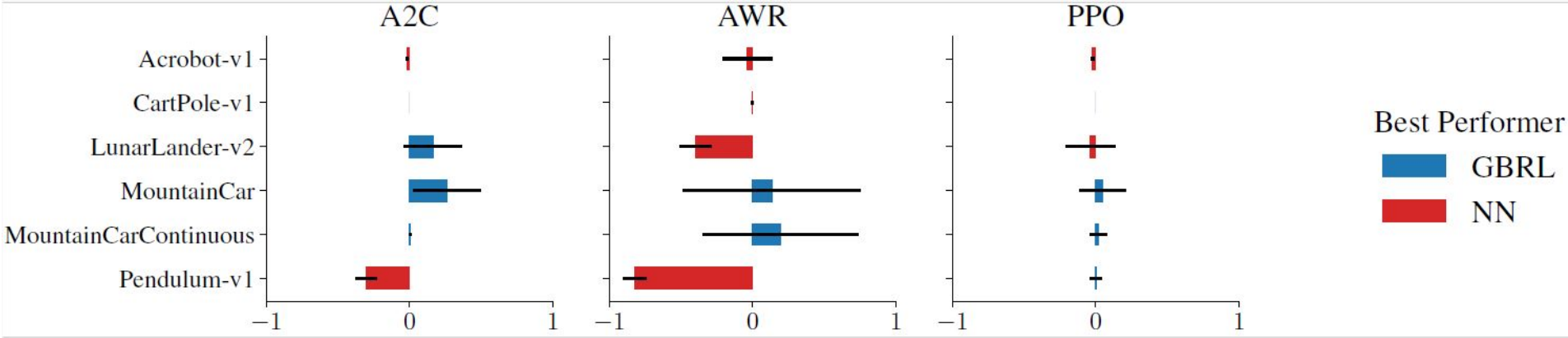
3. Benefits in Categorical Domains: Do the benefits of GBT in supervised learning transfer to the realm of RL?

4. Comparison to Traditional GBT libraries: Can we use traditional GBT libraries instead of GBRL for RL tasks?

5. Evaluating the shared AC architecture: How does sharing the tree structure between the actor and the critic impact performance?

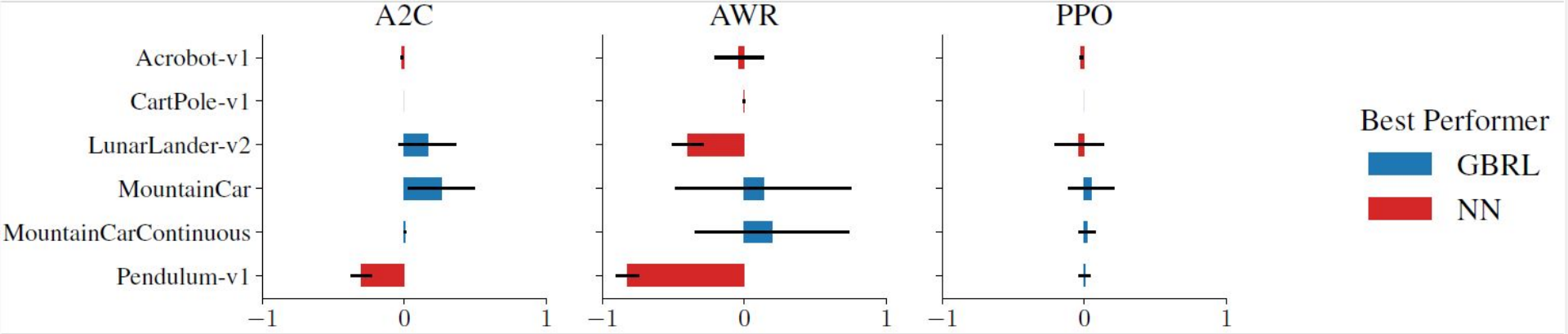
Experiments Continuous-Control and Box2D

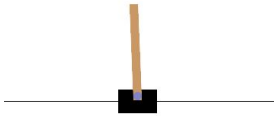
Classic Environments

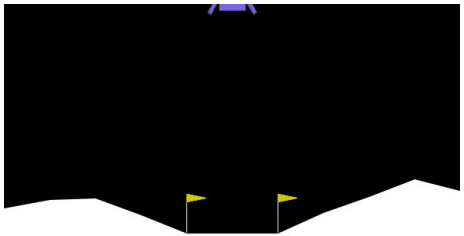


https://gymnasium.farama.org/environments/box2d/

Experiments Continuous-Control and Box2D

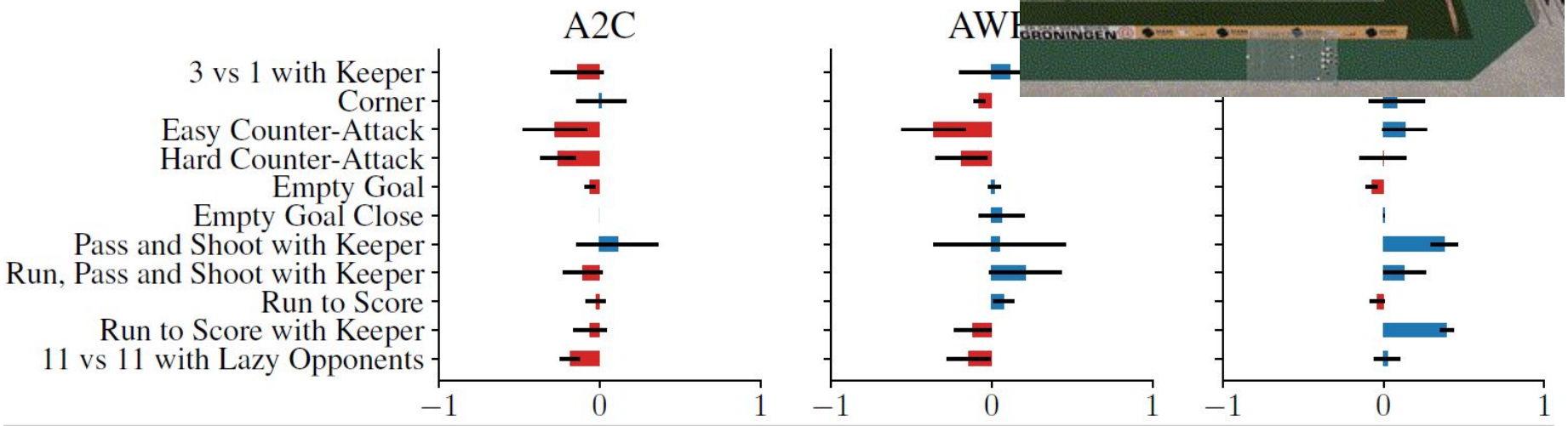






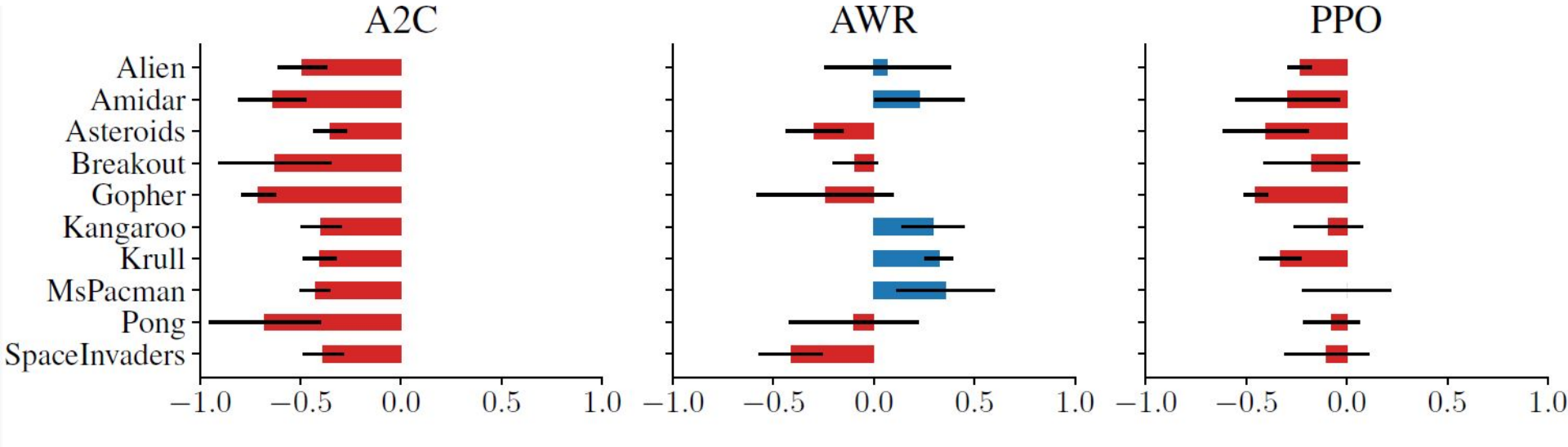
Experiments High-Dimensional Vectorized Environments

Structured dataset: Football Academy



Experiments High-Dimensional Vectorized Environments

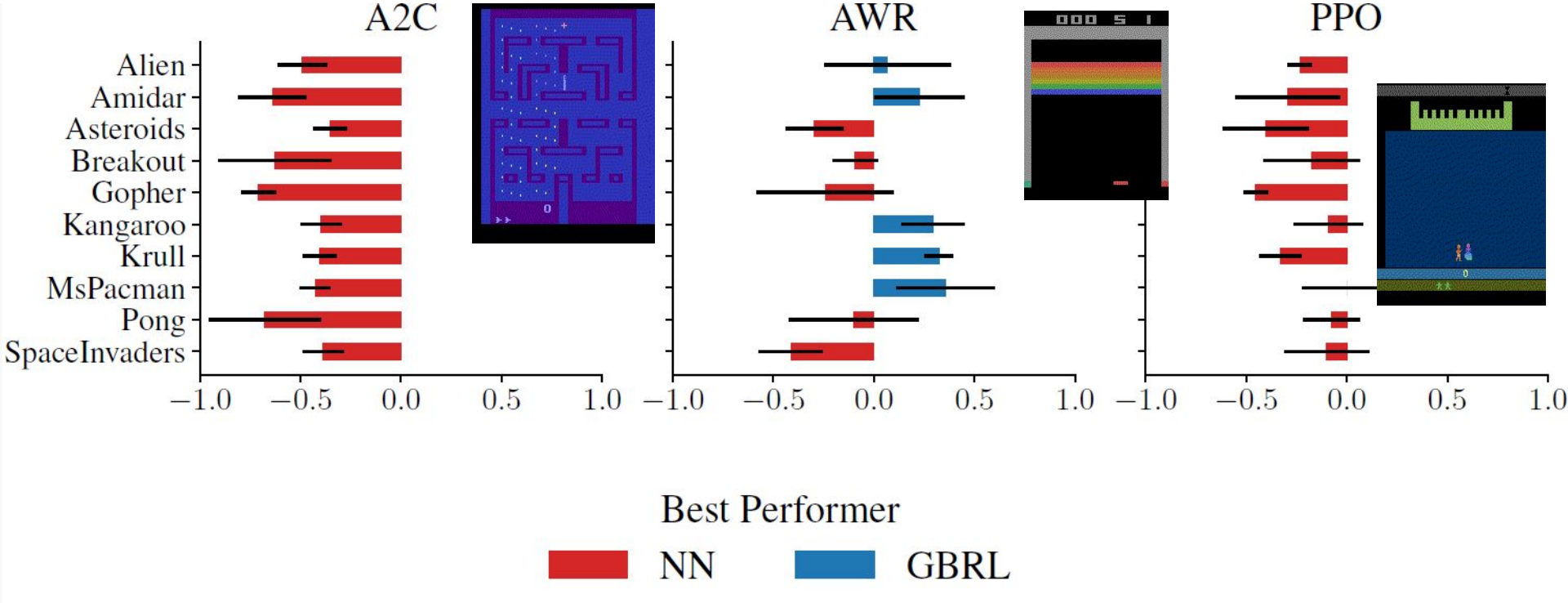
Unstructured dataset: Atari-ramNoFrameskip-v4 environments



https://gymnasium.farama.org/environments/box2d/

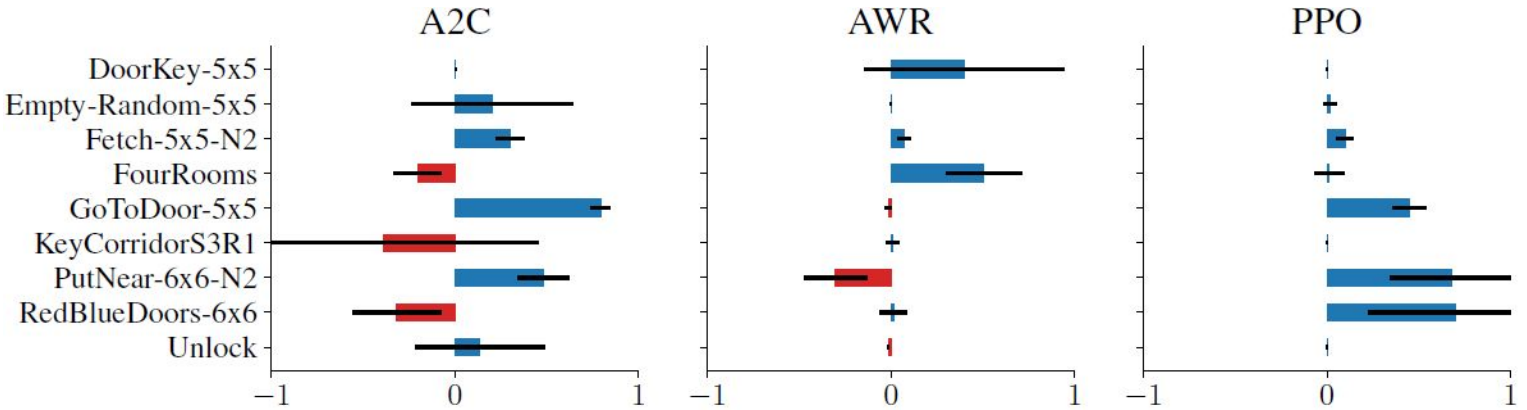
Experiments High-Dimensional Vectorized Environments

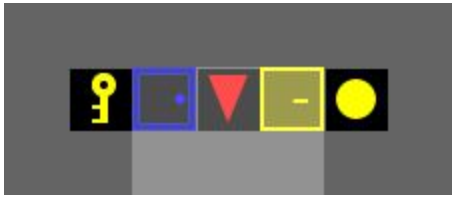
Unstructured dataset: Atari-ramNoFrameskip-v4 environments

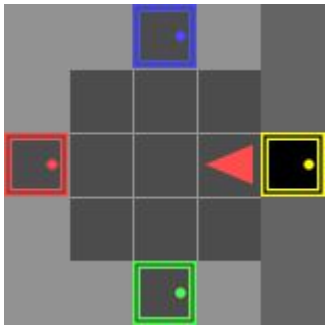


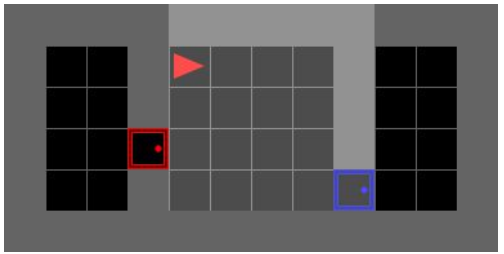
Experiments Categorical Environments

MiniGrid environments

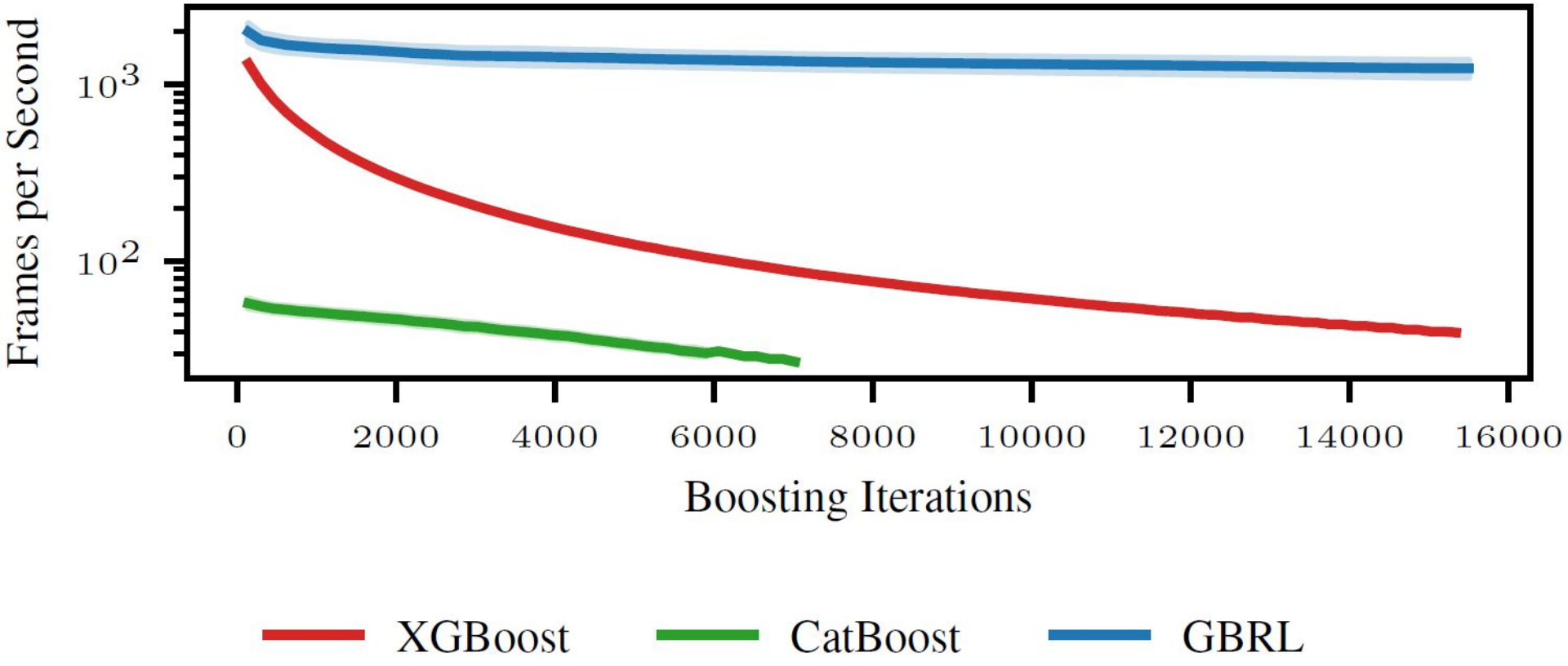






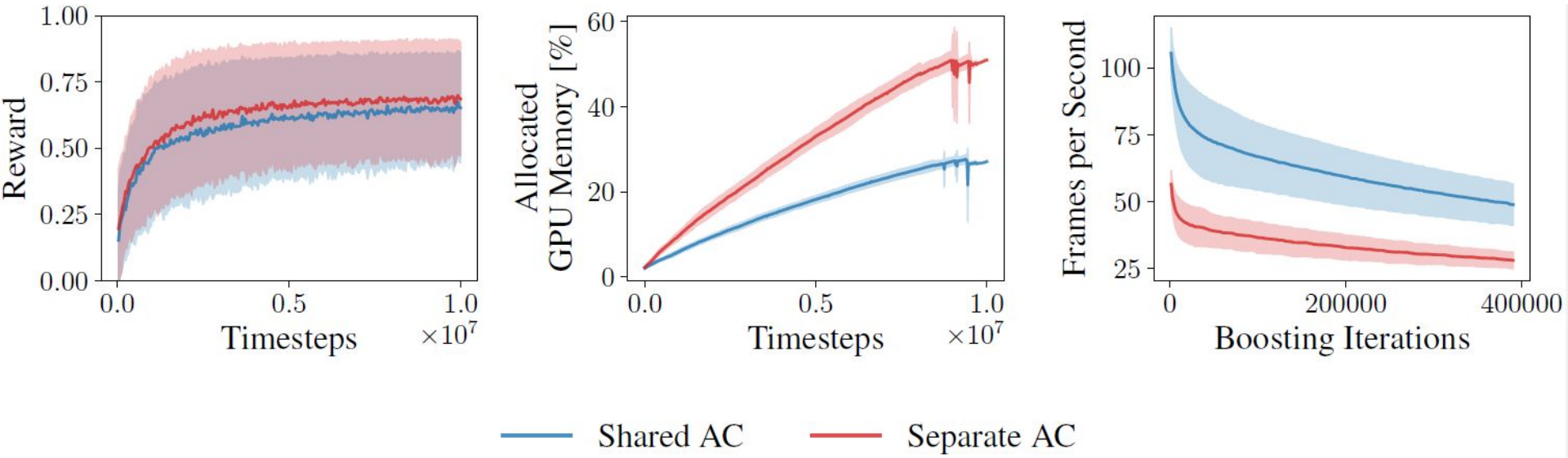


Experiments GBRL vs Traditional GBT Libraries



Experiments Evaluating the shared AC architecture

MiniGrid environments



Limitations and Conclusion

PPO GBRL vs PPO NN

1. **Gradient Boosting Trees for RL**

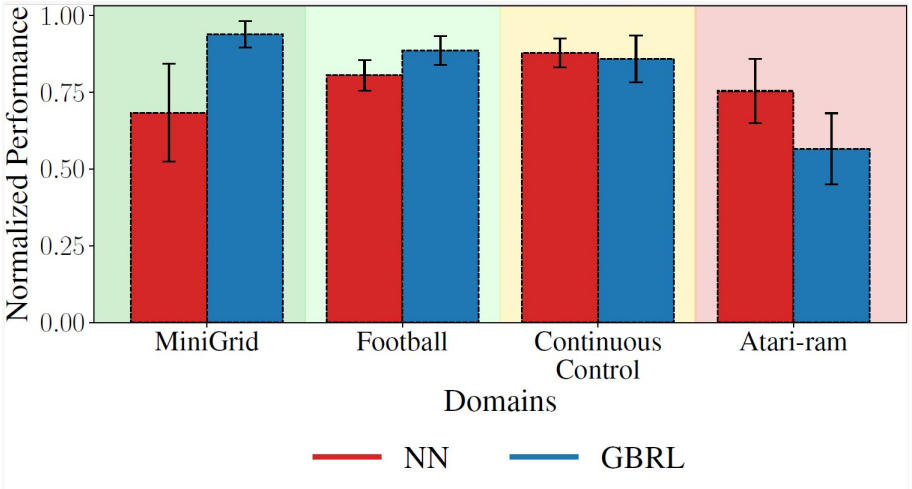
   In categorical tasks GBRL outperforms
   but need numerous updates, size increased

2. **Tree-based Actor-Critic architecture**

   GBRL reduce memory and run time than GBT
   Good at complex, ***yet structured*** environment

3. **Modern GBT-based RL library**

   provide CUDA-based hardware-accelerated GBT framework integrated in Stable-baseline3



GBRL library : https://github.com/NVlabs/gbrl

THANKS!!